



# Dominio y Puertos/Adaptadores

Cómo el estilo Hexagonal protege las reglas de la adopción

Taller Estilos Arquitectónicos y Stack Tecnológico · FAVM · Grupo 2

# El Dominio



## Refugio

Publica animales.

## Animal

disponible → postulado  
→ adoptado

## Adoptante

Postula. Se identifica por  
su email.

## Postulación

pendiente → aprobada /  
rechazada

**Reglas de negocio (viven en el núcleo, no en FastAPI ni en Mongo)**

**1** No se postula a un animal que no esté disponible

`Animal.validar_postulable()`

**2** Una sola postulación pendiente por adoptante y animal

`Postular + existe_pendiente()`

**3** Al aprobar: animal adoptado y las demás se cierran solas

`AprobarPostulacion`

# Ciclo de Vida



## Animal



## Postulación



Las transiciones inválidas las impide la propia entidad: ningún canal de entrada se las puede saltar.

# Puertos



```
class AnimalRepository(Protocol):
    async def obtener(self, animal_id)
        -> Animal | None
    async def listar(self, *, estado,
        especie, refugio_id, pagina, tamano)
    async def guardar(self, animal) -> Animal

class PostulacionRepository(Protocol):
    async def existe_pendiente(adoptante_id,
        animal_id) -> bool          # regla 2
    async def listar_por_animal(animal_id,
        *, estado)                  # regla 3
    ...
```

- Interfaces definidas por el núcleo (typing.Protocol)
- Dicen QUÉ necesita el caso de uso, no CÓMO
- Tipado estructural: se cumplen sin heredar

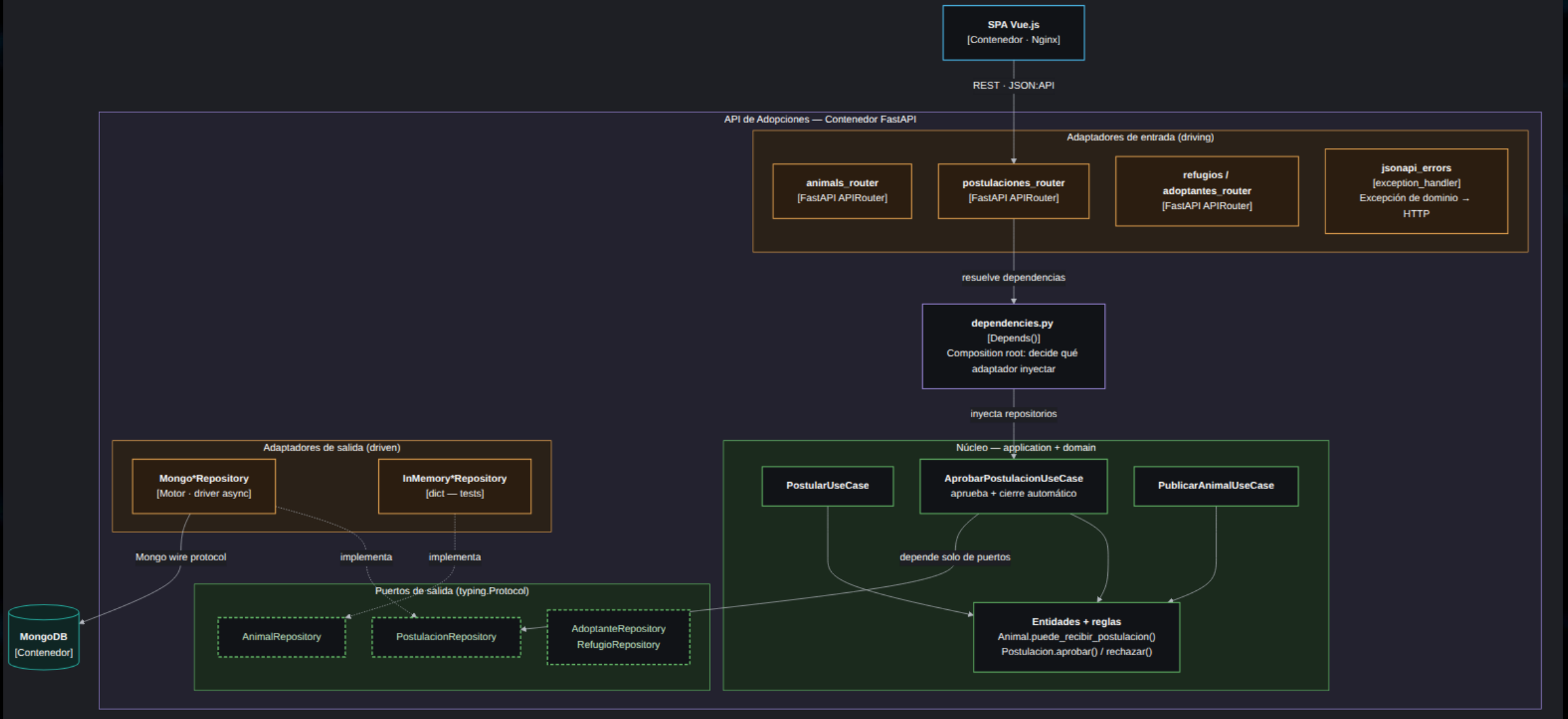
- 4 puertos pequeños: Animal, Postulación, Refugio, Adoptante
- Solo métodos que algún caso de uso usa (ISP, YAGNI)
- Ubicación: `app/application/ports/`

El dominio no importa FastAPI, Motor ni pymongo: una prueba automática lo verifica.

# Adaptadores



Sistema de Adopción de Mascotas — C4 Nivel 3 · Componentes de la API (Hexagonal)



# Aprobar: de HTTP al Dominio



1

**Router**

PATCH /v1/postulaciones/{id} valida el documento JSON:API (Pydantic, borde)

2

**Depends()**

dependencies.py inyecta los repositorios concretos

3

**Caso de uso**

AprobarPostulacion: postulacion.aprobar() y animal.marcar\_adoptado()

4

**Puertos**

guarda animal → aprobación → cierra las demás pendientes (en ese orden)

5

**Respuesta**

200 con el animal y las cerradas en included; o 409 si ya estaba resuelta

# Evidencia: Testabilidad



157

pruebas en total

100 %

cobertura del núcleo

≈ 2 s

suite unitaria

0

servicios externos necesarios

```
pytest-cov · cobertura del núcleo (domain + application) · sin red

$ docker run --network none ... # sin red: no hay MongoDB ni servidor alcanzable
$ python -m pytest tests/unit --cov=app/domain --cov=app/application --cov-report=term-missing
..... [100%]

----- coverage: platform linux, python 3.12.14-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
app/application/__init__.py         0      0  100%
app/application/ports/__init__.py    2      0  100%
app/application/ports/repositories.py 20      0  100%
app/application/use_cases/__init__.py  5      0  100%
app/application/use_cases/adoptantes.py 19      0  100%
app/application/use_cases/animales.py 26      0  100%
app/application/use_cases/postulaciones.py 72      0  100%
app/application/use_cases/refugios.py 16      0  100%
app/domain/__init__.py              0      0  100%
app/domain/entities.py             107     0  100%
app/domain/exceptions.py             6      0  100%
-----
TOTAL                               273     0  100%

78 passed in 2.13s
```

- Puertos mockeados: AsyncMock(spec=Puerto)
- Se verifica la interacción: un 409 nunca llega a guardar()
- docker run --network none: sin Mongo ni servidor
- Mismo caso de uso con in-memory, mocks y Mongo

# Decisiones y Costos



## Decisiones

- Pydantic valida formato en el borde; las reglas, en el dominio (ADR-006)
- Puertos como Protocol; el concreto se inyecta con Depends() (ADR-007)
- IDs como str en el dominio; el adaptador traduce ObjectId
- 'postulado' sigue aceptando postulaciones
- Excepción de dominio → HTTP en un solo archivo

## Costos asumidos

- Más archivos y mapeos que un CRUD (4 entidades)
- Indirección: hay que saltar de la interfaz a la implementación
- Validación en dos capas (borde y dominio)
- Excepciones en la periferia: un singleton, herencia en \_ConRepositorios

Se justifica por la regla de cierre automático: la corrección de las reglas pesa más que la simplicidad.